Por que funções são tão importantes em JavaScript?

Antes de entrar em objetos, arrays, DOM e APIs, o aluno precisa entender funções porque elas aparecem em praticamente tudo:

- em métodos de objetos
- em métodos de arrays como map, filter, forEach
- em eventos como onclick
- em temporizadores como setTimeout
- em requisições assíncronas
- em callbacks
- em promises e async/await

Objetivos da aula

Ao final desse conteúdo, o aluno deve ser capaz de:

- entender o que é uma função e para que serve;
- declarar funções de formas diferentes em JavaScript;
- usar parâmetros e retorno corretamente;
- perceber a diferença entre executar e referenciar uma função;
- entender escopo local e global;
- compreender que função pode ser tratada como valor;
- usar função como argumento de outra função;
- entender base de callback;
- perceber a ligação entre funções e os conteúdos futuros.

O que é uma função?

Uma função é um bloco de código criado para executar uma tarefa.

Exemplo simples:

```
function saudacao() {
    console.log("Olá, mundo!");
}
```

Para executar:

```
saudacao();
```

Cuidado!!!

Muitos alunos acham que “criar a função” já faz ela rodar.

Então vale destacar:

```
function saudacao() {
    console.log("Olá, mundo!");
}
```

Aqui a função foi definida, mas ainda não executada.

Só executa quando chamamos:

```
saudacao();
```

Estrutura básica da função

Sintaxe

```
function nomeDaFuncao(parametros) {
    // bloco de código
    return valor;
}
```

Partes:

- function → palavra-chave
- nomeDaFuncao → identificador
- (parametros) → entradas
- { } → corpo
- return → saída da função

Função sem parâmetro e sem retorno

Essa função apenas executa algo.

Exemplo simples:

```
function mostrarMensagem() {
    console.log("Bem-vindo ao JavaScript!");
}
```

Uso:

```
mostrarMensagem();
```

Função com parâmetro

Essa função apenas executa algo.

Exemplo simples:

```
function cumprimentar(nome) {
    console.log("Olá, " + nome);
}
```

Uso:

```
cumprimentar("Ana");
cumprimentar("Carlos");
```

Função com retorno

Essa função apenas executa algo.

Exemplo simples:

```
function somar(a, b) {
    return a + b;
}
```

Uso:

```
let resultado = somar(10, 5);
console.log(resultado);
```

Diferença essencial: mostrar ≠ retornar

Exemplo 1 — apenas mostra:

```
function somar1(a, b) {
    console.log(a + b);
}
```

Exemplo 2 — retorna:

```
function somar2(a, b) {
    return a + b;
}
```

Exemplo:

```
let x = somar2(2, 3);
let y = x * 10;
console.log(y);
```

Na primeira:

- o valor aparece na tela;
- mas não fica disponível para ser reutilizado.

Na segunda:

- o valor pode ser guardado em variável;
- pode ser usado em outra conta;
- pode ser passado para outra função.

Parâmetros e argumentos

É bom diferenciar:

parâmetro: nome usado na definição da função

argumento: valor real passado na chamada

```
function multiplicar(a, b) { // a e b = parâmetros
    return a * b;
}

multiplicar(3, 4); // 3 e 4 = argumentos
```

Parâmetro padrão

Muito útil e moderno.

```
function saudar(nome = "visitante") {
    console.log("Olá, " + nome);
}
```

Uso:

```
saudar("Maria");
saudar();
```

Saídas:

```
Olá, Maria
Olá, visitante
```

O que acontece se faltar ou sobrar argumento?

Muito útil e moderno.

```
function somar(a, b) {
    return a + b;
}

console.log(somar(10));
```

Nesse caso, b será undefined.

Resultado:

NaN

Função como organização do programa

Funções ajudam a dividir o problema em partes menores.

Sem função:

```
let n1 = 8;
let n2 = 6;
let media = (n1 + n2) / 2;
console.log(media);

let a = 9;
let b = 7;
let media2 = (a + b) / 2;
console.log(media2);
```

Com função:

```
function media(x, y) {
    return (x + y) / 2;
}

console.log(media(8, 6));
console.log(media(9, 7));
```

Escopo: variável local e global

Funciona:

```
let mensagem = "Global";

function teste() {
    let nome = "Ana";
    console.log(nome);
    console.log(mensagem);
}

teste();
console.log(mensagem);
```

Mas:

```
function teste() {
    let nome = "Ana";
}

console.log(nome);
```

Dá erro, porque nome só existe dentro da função.

A função cria um “ambiente próprio”.

Formas de declarar função

Function Declaration

```
function somar(a, b) {
    return a + b;
}
```

Function Expression

```
const somar = function(a, b) {
    return a + b;
};
```

Arrow Function

```
const somar = (a, b) => {
    return a + b;
};
```

Versão curta:

```
const somar = (a, b) => a + b;
```

Diferença entre declarar e guardar função em variável

```
function ola() {
    console.log("Olá");
}

const oi = function() {
    console.log("Oi");
};
```

As duas funcionam, mas perceba que no segundo caso a função está sendo tratada como valor.

Função como valor

Esse é um dos conceitos mais importantes de JavaScript.

```
const mensagem = function() {
    console.log("Função guardada em variável");
};

mensagem();
```

A função pode ser:

- **armazenada em variável;**
- **passada para outra função;**
- **retornada por outra função.**

Esse ponto constrói a ponte para:

- **callbacks**
- **eventos**
- **métodos de array**
- **async**

Diferença entre minhaFuncao e minhaFuncao()

Outro ponto crítico.

```
function teste() {
    console.log("Executou");
}
```

teste → referência da função

teste( ) → execução da função

Exemplo didático:

```
console.log(teste);   // mostra a função
teste();              // executa a função
```

Função chamando outra função

Boa forma de mostrar modularização

```
function dobro(n) {
    return n * 2;
}

function mostrarDobro(valor) {
    console.log(dobro(valor));
}

mostrarDobro(5);
```

Função com decisão interna

Função pode encapsular regra de negócio

```
function verificarPar(numero)
{
    if (numero % 2 === 0) {
        return "par";
    } else {
        return "ímpar";
    }
}
```

Função retornando booleano

```
function ehMaiorDeIdade(idade) {
    return idade >= 18;
}
```

Uso:

```
if (ehMaiorDeIdade(20)) {
    console.log("Pode entrar");
}
```

Esse modelo ajuda muito na organização do raciocínio

Funções e reutilização de regras

```
function validarNota(nota) {
    return nota >= 0 && nota <= 10;
}
```

Uso:

```
let nota = 8;

if (validarNota(nota)) {
    console.log("Nota válida");
} else {
    console.log("Nota inválida");
}
```

Esse modelo ajuda a criar funções utilitárias antes de mexer com DOM e formulários.

Introdução a callback

Uma função pode receber outra função como argumento.:

```
function executar(funcao) {
    funcao();
}

function mensagem() {
    console.log("Executando a função
recebida");
}

executar(mensagem);
```

Entenda!!!

Não foi passado o resultado da função.
Foi passada a própria função.

Esse é o alicerce de:

- forEach
- map
- filter
- addEventListener
- setTimeout
- fetch(...).then(...)

Exemplo simples de callback com parâmetros

Uma função pode receber outra função como argumento.:

```
function calcular(a, b, operacao) {
    return operacao(a, b);
}

function somar(x, y) {
    return x + y;
}

function multiplicar(x, y) {
    return x * y;
}

console.log(calcular(4, 5, somar));
console.log(calcular(4, 5,
multiplicar));
```

Funções anônimas

Função sem nome

```
const soma = function(a, b) {
    return a + b;
};


setTimeout(function() {
    console.log("Olá");
}, 1000);
```

São muito usadas quando a função será usada em um contexto específico